

Aula 41 - Preparatória do Projeto de SA

Descrição: Nesta aula prática, as equipes de estudantes irão apresentar a prévia do projeto de SA. Cada equipe terá um tempo para realizar a apresentação e receber o feedback dos professores.

Introdução: Vamos estruturar esse passo a passo para o software do projeto de SA, mantendo a arquitetura robusta e a stack tecnológica nativa (PHP 8.x, MySQL, JavaScript ES6, HTML5/CSS3).

Aqui está o roteiro de desenvolvimento, organizado com as entregas e configurações esperadas fase:

Fase 1: Levantamento e Engenharia de Requisitos

Nesta fase, traduzimos as necessidades do negócio em regras claras.

- **Documentos a entregar:** Documento de Especificação de Requisitos (SRS).
- **Ações:**
 - Mapear as entidades principais do sistema genérico.
 - Definir a matriz de Controle de Acesso Baseado em Papéis (RBAC), determinando quem pode ler, criar ou editar dados.
 - Estabelecer os fluxos de operação padrão (casos de uso).

Fase 2: Modelagem de Dados e Integridade

Preparação da camada de persistência com foco em auditoria.

- **Documentos a entregar:** Diagrama Entidade-Relacionamento (DER), Dicionário de Dados.
- **Arquivos/Configurações:** `database_schema.sql`.
- **Ações:**
 - Criar as tabelas no MySQL.
 - Adicionar uma coluna `status` (ex: ativo/inativo) em todas as tabelas principais para garantir o **Soft Delete** (baixa lógica).
 - Configurar chaves estrangeiras (Foreign Keys) para garantir integridade referencial.

Fase 3: Infraestrutura e Arquitetura do Back-end

Configuração da base do projeto em PHP nativo utilizando padrões de mercado.

- **Documentos a entregar:** Relatório Técnico de Arquitetura.
- **Arquivos/Configurações vitais:**
 - `composer.json`: Configurado exclusivamente para o Autoloading de classes usando a norma **PSR-4** (mapeando o namespace do projeto, ex: `"App\\": "src/"`).
 - `config/database.php`: Arquivo com as credenciais do banco (isolado do diretório público).
 - `public/index.php`: O **Front Controller**. Único ponto de entrada do sistema.
 - `public/.htaccess` (se usar Apache): Configurado para redirecionar todas as requisições para o `index.php` e bloquear acesso a outras pastas.
- **Ações:**
 - Criar a estrutura de pastas separando o que é público do que é regra de negócio (ex: `/public`, `/src/Controllers`, `/src/Models`, `/src/Views`).
 - Desenvolver o roteador central no Front Controller.

Fase 4: Desenvolvimento da Camada de Domínio (MVC)

Implementação da lógica de negócios e conexão segura com o banco.

- **Arquivos:** Classes dentro de `src/Models/` e `src/Controllers/`.

- **Ações:**

- **Conexão Unificada:** Criar uma classe estática ou Singleton de conexão PDO para ser consumida pelos Controllers/Models.
- **Princípios SOLID:** Garantir que as classes tenham responsabilidade única (ex: separar validação de dados da gravação no banco).
- **Transações ACID:** Nas operações que alteram mais de uma tabela, implementar blocos `try/catch` utilizando `$pdo->beginTransaction()`, `$pdo->commit()` e `$pdo->rollBack()`.

Fase 5: Interface de Usuário (Front-end)

Criação das telas de forma nativa e leve.

- **Arquivos:** `public/css/style.css`, `public/js/app.js`, e arquivos de visualização em `src/Views/` (arquivos `.php` que mesclam HTML5 com as variáveis do Controller).
- **Ações:**
 - Estruturar o HTML5 de forma semântica.
 - Aplicar CSS3 para responsividade (preparado para desktops e tablets).
 - Utilizar JavaScript Vanilla (ES6) para interações assíncronas (como validações de formulário ou consumo de rotas internas via `fetch API`) sem sobrecarregar o cliente.

Fase 6: Testes e Implantação

Validação da segurança e entrega final.

- **Documentos a entregar:** Manual do Usuário, Termo de Homologação.
- **Ações:**
 - Testar se o isolamento de diretórios está funcionando (tentar acessar a pasta `src/` diretamente pela URL e garantir que retorna erro).
 - Testar as restrições do RBAC com diferentes usuários.
 - Realizar o deploy em ambiente de produção (servidor Linux/Apache ou Nginx).